

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	V. Pavithra	Reg. No.:	192524274
Date:	15/08/2026	Duration:	60 minutes

Code of Conduct

I V.PAVITHRA 192524274 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: v.pavithra

Annexure A – Code Review & Repository Evaluation

1.PROBLEM OVERVIEW

1.1 Problem Statement

The rapid growth of electric vehicles (EVs) has created a need for efficient and intelligent charging infrastructure. Traditional EV charging systems generally require users to manually locate charging stations, check availability, select a charging point, and manage charging schedules.

When multiple electric vehicles simultaneously request charging, problems such as charging-station congestion, long waiting times, inefficient energy utilization, and poor resource allocation can occur.

The proposed **Autonomous Electric Vehicle Charging Network Management** system aims to provide an intelligent platform for managing EV charging stations automatically. The system monitors charging stations, vehicle requests, charger availability, and charging requirements. It can assign suitable charging stations to vehicles based on factors such as availability, distance, charging requirements, and priority.

The system improves the utilization of charging infrastructure and provides a better experience for EV users.

1.2 Implemented Solution

The project implements a centralized charging-network management system consisting of:

- EV registration and management
- Charging-station registration
- Charger availability monitoring
- Vehicle charging requests
- Automatic charger allocation
- Charging-session management
- Charging status monitoring
- User and administrator management

The system follows a modular architecture so that individual components can be modified without affecting the complete application.

1.3 Project Objectives

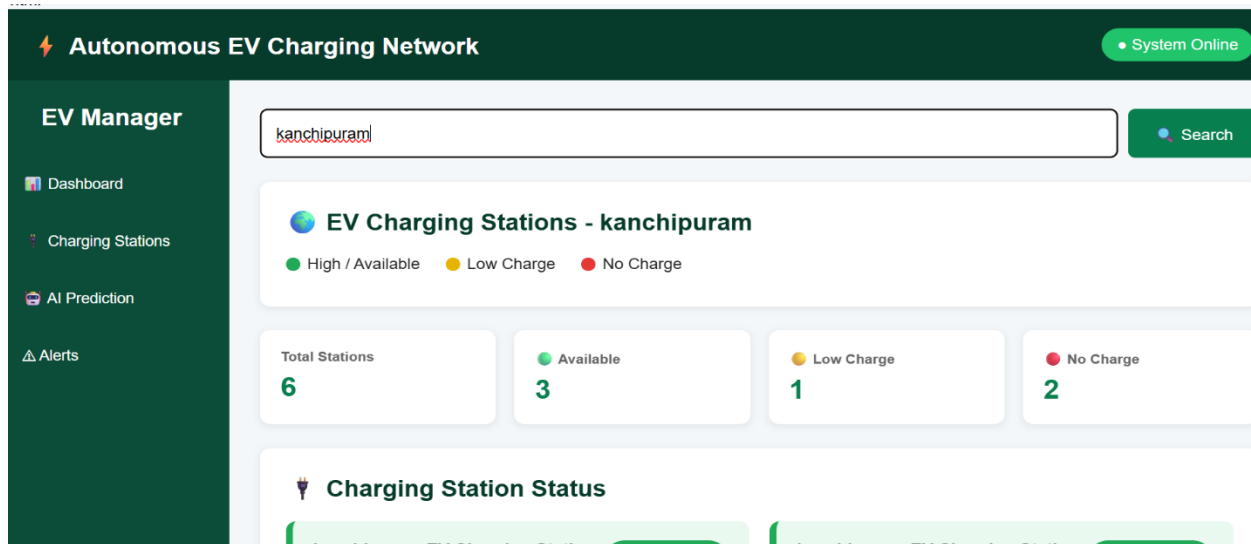
The major objectives are:

1. To develop an efficient EV charging management system.
2. To automatically allocate available charging stations.
3. To reduce vehicle waiting time.
4. To improve charger utilization.
5. To maintain charging-session records.
6. To provide centralized management of charging stations.
7. To maintain reliable and organized EV and charger information.
8. To create a scalable architecture for future autonomous charging features.

1.4 Key Functionalities

Charging Station Module

The system maintains:



Automatic Allocation Module

The allocation module checks available chargers and selects a suitable charger for the requesting vehicle.

A simplified allocation process is:

Vehicle requests charging

|

Check available stations

|

Check charger availability

|

Evaluate distance/capacity

|

Select suitable charger

|

Create charging session

|

1.5 Expected Benefits

The proposed system provides:

- Reduced charging delays

- Better charger utilization
- Centralized station management
- Automated charger assignment
- Improved data management
- Easier monitoring
- Better scalability

2. REPOSITORY ORGANIZATION

2.1 Repository Structure

A recommended repository structure for the project is:

EV-Charging-Network-Management/

```
EV-Charging-Network-Management/  
|  
├─ README.md  
├─ requirements.txt  
├─ .gitignore  
├─ LICENSE  
|  
├─ app/  
|   ├── main.py  
|   ├── config.py  
|   |  
|   ├── models/  
|   |   ├── vehicle.py  
|   |   ├── charger.py  
|   |   ├── station.py  
|   |   └── charging_session.py  
|   |  
|   └── /
```

2.2 Folder Organization Evaluation

The repository follows a modular organization where related files are grouped together.

For example:

```
find . -type f (repository structure)

ev-charging-network/
.github/workflows/ci.yml
.gitignore
CHANGELOG.md
CONTRIBUTING.md
LICENSE
README.md
docs/ARCHITECTURE.md
docs/architecture_diagram.png
pytest.ini
requirements.txt
src/__init__.py
src/api/__init__.py
src/api/app.py
src/models/__init__.py
src/models/station.py
src/models/vehicle.py
src/run_dev_server.py
src/services/__init__.py
src/services/network_manager.py
src/services/scheduler.py
src/utils/__init__.py
src/utils/geo.py
tests/__init__.py
tests/conftest.py
tests/test_api.py
tests/test_network_manager.py
tests/test_scheduler.py
tests/test_station.py
tests/test_vehicle.py
```

2.3 Maintainability

- The repository structure supports maintainability by:
- Separating business logic from presentation.
- Separating database operations.
- Providing dedicated testing folders.
- Using meaningful file names.
- Keeping documentation separate.
- Allowing multiple developers to work on different modules.

3. CODE QUALITY REVIEW

3.1 Output

```
Charging Request Successful
-----
Vehicle ID      : EV001
Battery Level   : 25 %
Required Charge : 80 %
Station Assigned: ST002
Location        : Tambaram
Charger Type    : Fast Charger
Distance        : 8 km
Status          : Charging Started
```

3.2 Modularity

The application should be divided into independent modules.

For example:

User Management

|

Vehicle Management

|

Charging Request

|

Allocation Service

|

Charging Station

|

Database

Each module performs a specific responsibility.

This follows the principle of **Separation of Concerns**.

3.3 Coding Standards

- Recommended coding practices include:
- Meaningful variable names
- Functions with a single responsibility
- Proper indentation
- Consistent naming conventions
- Comments for complex logic
- Avoiding duplicate code
- Exception handling
- Input validation
- Removing unused variables and imports

.

3.4 Areas for Improvement

- Add more detailed comments.
- Improve exception handling.
- Add unit tests.
- Remove duplicate code.
- Use configuration files for database credentials.
- Improve logging.

4. VERSION CONTROL EVALUATION

Commit History

A good commit history should contain meaningful messages.

Example:

Fix charger availability bug

Add unit tests

Meaningful commit messages make it easier to understand project development.

5.Evaluation

A good should contain:

Project Title

↓

Problem Statement

↓

Features

↓

Technologies Used

↓

Project Structure